Capstone Project Proposal: Academic Information and Management System

What will the program do?

The Academic Information and Management System will be a program written entirely in C++ programming language and will be console-based. The program is designed to help academic administration by managing the core institutional entities like the student records, enrolling them in different courses, and updating their grades. The key features of the program include the following

- Record Management: Users will be able to add, update, review and delete records on students, instructors and courses.
- Enrolling and Grading: Students can be enrolled in courses at the beginning of term and instructors will be able to update the students' grades.
- Automated Calculation: Through the program instructors can automatically calculate the cumulative and semester GPA based on the grade the student received and the credit hours.
- Reporting and Searching: The program will be able to generate students' performance reports and Transcripts along with teaching loads of the faculty and will have searching and sorting capabilities for easy retrieval of data.

Who will use the program? Identify all intended users.

User Type	Role	Responsibility
Primary	Administrative Staff	Record keeping all the official information of students, instructors and courses, generate transcripts
Secondary	Instructors	Entering the students end of term grades of the course they teach

Secondary	Advisory Staff	Keeping track of student's progress and enrolling them on courses as needed

Why will the program be of use to the intended users? What data will the user input into your program?

The program will help the institution in transitioning from a manual paper-based record keeping to a much more reliable and faster console interface. The general issue of accidental mistakes like entering the name of the student incorrectly or assigning the multiple students the same ID, and mismatched grades will be minimized significantly through the use of this program. The program through its interconnected records ensures that the updates such as student dropping a course or instructors updating the grades are implemented accurately all throughout the system rather than having a user update one by one on each spreadsheet. The struggle for manual computation of grades will be simplified as instructors and advisors can just update the grades of the students and the program will automatically calculate it for them. The searching and sorting mechanism will further help the users in pinpointing their required records instantly like finding a student transcript or trying to sort the students with highest GPA rather than spending hours on paper files or spreadsheets.

What data will be required by your program?

The program will require the following data:

Data type	Data required
Person data (Student and Instructor)	• ID number • Full name • Date of birth • Contact information
Student-specific data	• Major / Program • History of enrollment • List of enrolled courses

	• Grades per course • Cumulative and semester GPA
Instructor-specific data	• Department • List of assigned courses • Teaching load
Course data	• Course code • Course title • Credit hours • Term offered • Instructor assigned
Enrollment data	• Student ID • Course ID • Term • Grade earned • Enrollment status

Abstract Data Types

In my project, the Person, Student, Instructor, Course and Enrollment are defined as abstract data types. For handling the identity data which is shared by all the individuals in the system like ID, name and date of birth, a Person class is defined. To expose this data safely accessor and mutator methods must be implemented. Both the student and instructor in addition to inheriting from the Person class have their own unique private data. The GPA of the student and the enrollment history will be contained in Student class while assigned courses will belong in the instructor class. To retrieve and update those data each class will have their own public operations. Separate from Person hierarchy Course and Enrollment are defined as separate abstract data types. Course will encapsulate the courses that are offered while enrollment will encapsulate the relation between the student and instructor along with grades earned. Only through these public methods does the interaction from modules like record management, grading and reporting takes place and never directly through their internal data as each class hides its internal representation.

Advanced Concepts Implementation

- Inheritance: While working with classes the program will encounter the problem of redundancy to solve it inheritance will be used. Without the use of inheritance, I will have to create identical member variables and accessor methods for each user type which will lead to redundancy. The subclasses like Student and Instructor have shared attributes like name, ID and date of birth by using inheritance it can be inherited from a base class, Person which reduces redundancy.
- Sorting Algorithms: Through sorting the user will be able to organize their desired records on demand like ranking students by GPA or arrange them alphabetically. In this program, a merge sort algorithm will be implemented from scratch which will be working through the overloaded less than operator (<) on the classes so that depending on whatever the user selects like GPA or name they will be able to sort using the same logic.
- Polymorphism: For displaying outputs through a common interface based on whether it is a student or instructor object, polymorphism with a virtual function will be used. In Person class the virtual function is declared which will be overridden separately in each subclass. Runtime polymorphism is enabled through a generic collection of base pointers and as the program loops through the collection, and whichever entity type it gets pointed to correctly invokes the specific customized logic.
- Overloaded operators: To enable direct print of object details on console using cout stream insertion operator (<<) will be overloaded for the student, instructor and courses classes. To further simplify search and validation process the program will overload the equality operator (==) which will help in easy check for two objects that match directly. To further streamline the implementation of sorting algorithms less than operator (<) will be implemented to compare the GPAs and IDs of the students directly.